

Compte rendu – Jeu Python : *Labyrinthe*

1. Présentation générale du projet “LABYRINTHE DE LA PRINCESSE”

Dans le cadre d’un travail pratique en programmation Python, nous avons réalisé un petit jeu intitulé « **Labyrinthe de la Princesse** », utilisant la bibliothèque **Pygame**.

Le principe du jeu est simple : une princesse se retrouve perdue dans un labyrinthe et doit retrouver son chemin jusqu’à la sortie. Le joueur contrôle la princesse et doit la guider en évitant les murs.

Le design du jeu est volontairement **rose et girly**, afin de correspondre à un thème esthétique spécifique. Le visuel est inspiré d’un style pixel-art/jeu simple réalisé en Python.

2. Objectifs du projet

Ce projet avait plusieurs objectifs pédagogiques :

- Manipuler la bibliothèque **Pygame**
- Comprendre la gestion d’événements (clavier, boucle de jeu)
- Générer et afficher un **labyrinthe** représenté sous forme de grille
- Mettre en place une **détection des collisions**
- Gérer un personnage, son déplacement et une **condition de victoire**
- Structurer un jeu complet dans un seul fichier Python

3. Fonctionnement du jeu

a. Contrôles

- **Flèche Haut** : déplacer la princesse vers le haut
- **Flèche Bas** : déplacer la princesse vers le bas
- **Flèche Gauche** : aller à gauche
- **Flèche Droite** : aller à droite

La princesse ne peut pas traverser les murs du labyrinthe.

b. Objectif

Le joueur doit atteindre la case marquée **E (Exit)** dans le labyrinthe.

Lorsque la princesse arrive à la sortie, un message de félicitations apparaît.

4. Structure technique du programme

```
30 HEIGHT = ROWS * TILE_SIZE
31
32 # Couleurs (palette rose)
33 BG_COLOR = (255, 245, 228) # #fff5e4
34 WALL_COLOR = (238, 105, 131) # #ee6983
35 PLAYER_COLOR = (253, 108, 158) # #fd6c9e
36 PLAYER_OUTLINE = (162, 73, 103) # #a24967
37 EXIT_COLOR = (162, 73, 103) # #a24967
38 GRID_COLOR = (255, 196, 196) # #ffc4c4
39
40 # Vitesse de déplacement : 1 case par appui
41 MOVE_DELAY = 100 # ms entre deux déplacements si touche maintenue
42
43 # -----
44 # Fonctions utilitaires
45 # -----
46
47 def find_char_in_maze(char):
48     """Retourne la position (ligne, col) du caractère donné dans le labyrinthe."""
49     for row_idx, row in enumerate(MAZE):
50         col_idx = row.find(char)
51         if col_idx != -1:
52             return row_idx, col_idx
53     return None
54
55 def can_move_to(row, col):
56     """Vérifie si le joueur peut se déplacer sur la case (row, col)."""
57     if 0 <= row < ROWS and 0 <= col < COLS:
58         return MAZE[row][col] != "1"
59     return False
```

```

61 # -----
62 # Initialisation de Pygame
63 # -----
64 pygame.init()
65 screen = pygame.display.set_mode((WIDTH, HEIGHT))
66 pygame.display.set_caption("Labyrinthe de la Princesse (Python)")
67
68 clock = pygame.time.Clock()
69 font = pygame.font.SysFont("arial", 22, bold=True)
70 big_font = pygame.font.SysFont("arial", 40, bold=True)
71
72 # Position de départ (princesse)
73 start_row, start_col = find_char_in_maze("S")
74 player_row, player_col = start_row, start_col
75
76 # Position de la sortie
77 exit_row, exit_col = find_char_in_maze("E")
78
79 last_move_time = 0
80 game_finished = False
81
82 # -----
83 # Dessin
84 # -----
85
86 def draw_grid():
87     """Dessine la grille légère en fond."""
88     for x in range(0, WIDTH, TILE_SIZE):
89         pygame.draw.line(screen, GRID_COLOR, (x, 0), (x, HEIGHT))
90     for y in range(0, HEIGHT, TILE_SIZE):
91         pygame.draw.line(screen, GRID_COLOR, (0, y), (WIDTH, y))

```

```

86 def draw_grid():
90     for y in range(0, HEIGHT, TILE_SIZE):
91         pygame.draw.line(screen, GRID_COLOR, (0, y), (WIDTH, y))
92
93 def draw_maze():
94     """Dessine les murs et la sortie."""
95     for row in range(ROWS):
96         for col in range(COLS):
97             cell = MAZE[row][col]
98             x = col * TILE_SIZE
99             y = row * TILE_SIZE
100
101             if cell == "1":
102                 pygame.draw.rect(screen, WALL_COLOR, (x, y, TILE_SIZE, TILE_SIZE))
103             elif cell == "E":
104                 # Dessiner une case spéciale 'cœur / sortie'
105                 pygame.draw.rect(screen, BG_COLOR, (x, y, TILE_SIZE, TILE_SIZE))
106                 # Un petit cœur stylisé avec du texte "<3"
107                 exit_rect = pygame.Rect(x, y, TILE_SIZE, TILE_SIZE)
108                 pygame.draw.rect(screen, EXIT_COLOR, exit_rect, border_radius=10)
109                 text = font.render("<3", True, BG_COLOR)
110                 text_rect = text.get_rect(center=exit_rect.center)
111                 screen.blit(text, text_rect)
112
113 def draw_player(row, col):
114     """Dessine la princesse comme un personnage mignon."""
115     x = col * TILE_SIZE
116     y = row * TILE_SIZE
117     center_x = x + TILE_SIZE // 2
118     center_y = y + TILE_SIZE // 2

```

```

# Robe (cercle / ovale)
dress_rect = pygame.Rect(0, 0, TILE_SIZE - 8, TILE_SIZE - 10)
dress_rect.center = (center_x, center_y + 3)
pygame.draw.ellipse(screen, PLAYER_COLOR, dress_rect)

# Tête
head_radius = TILE_SIZE // 5
head_center = (center_x, center_y - TILE_SIZE // 4)
pygame.draw.circle(screen, (255, 220, 200), head_center, head_radius)

# Contour de la princesse
pygame.draw.ellipse(screen, PLAYER_OUTLINE, dress_rect, 2)
pygame.draw.circle(screen, PLAYER_OUTLINE, head_center, head_radius, 2)

# Petite couronne (triangle)
crown_width = TILE_SIZE // 3
crown_height = TILE_SIZE // 4
cx, cy = head_center
crown_points = [
    (cx - crown_width // 2, cy - head_radius), # gauche
    (cx, cy - head_radius - crown_height), # sommet
    (cx + crown_width // 2, cy - head_radius), # droite
]
pygame.draw.polygon(screen, PLAYER_COLOR, crown_points)
pygame.draw.polygon(screen, PLAYER_OUTLINE, crown_points, 2)

def show_victory_message():
    """Affiche un message de victoire au centre de l'écran."""
    msg1 = big_font.render("Bravo !", True, EXIT_COLOR)

```

```

46 def show_victory_message():
47     msg2 = font.render(
48         "La princesse a retrouvé son chemin ❤️", True, EXIT_COLOR
49     )
50
51     rect1 = msg1.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 20))
52     rect2 = msg2.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 20))
53
54     screen.blit(msg1, rect1)
55     screen.blit(msg2, rect2)
56
57 # -----
58 # Boucle principale du jeu
59 # -----
60 running = True
61 while running:
62     dt = clock.tick(60) # Limite à 60 FPS
63     current_time = pygame.time.get_ticks()
64
65     for event in pygame.event.get():
66         if event.type == pygame.QUIT:
67             running = False
68
69     keys = pygame.key.get_pressed()
70
71     if not game_finished:
72         # Gestion des déplacements (touche maintenue possible)
73         if current_time - last_move_time > MOVE_DELAY:
74             new_row, new_col = player_row, player_col
75
76             if keys[pygame.K_UP]:

```

```

178         if keys[pygame.K_UP]:
179             new_row -= 1
180         elif keys[pygame.K_DOWN]:
181             new_row += 1
182         elif keys[pygame.K_LEFT]:
183             new_col -= 1
184         elif keys[pygame.K_RIGHT]:
185             new_col += 1
186
187         if (new_row, new_col) != (player_row, player_col):
188             if can_move_to(new_row, new_col):
189                 player_row, player_col = new_row, new_col
190                 last_move_time = current_time
191
192         # Vérifier si la princesse est arrivée à la sortie
193         if player_row == exit_row and player_col == exit_col:
194             game_finished = True
195
196     # Dessin de la scène
197     screen.fill(BG_COLOR)
198     draw_grid()
199     draw_maze()
200     draw_player(player_row, player_col)
201
202     if game_finished:
203         show_victory_message()
204
205     pygame.display.flip()
206
207 # Quitter proprement

```

- Le labyrinthe est codé sous forme d'une **liste de chaînes de caractères**, où :
 - 1 = mur
 - 0 = chemin
 - S = position de départ
 - E = sortie
- Le personnage est dessiné avec des formes simples (cercles/ellipses) pour créer l'apparence d'une princesse.
- Le jeu utilise :
 - `pygame.display` pour la fenêtre
 - `pygame.event` pour gérer les touches
 - Une boucle principale limitée à **60 FPS**
 - Un système de temporisation pour éviter les déplacements trop rapides

5. Choix graphiques

Le thème graphique repose sur une palette de roses :

- #fd6c9e
- #ee6983
- #ffc4c4
- #fff5e4
- #a24967

Les murs, le sol, la grille, la princesse et la sortie utilisent des teintes différentes pour une ambiance cohérente et esthétique.

Un petit **cœur** marque la sortie du labyrinthe, renforçant le style mignon du jeu.

6. Résultats obtenus

Le jeu fonctionne correctement et permet :

- L’affichage du labyrinthe
- Le contrôle du personnage
- Le blocage lorsque la princesse rencontre un mur
- La détection de victoire
- Une ambiance visuelle agréable et adaptée au thème

Le projet remplit l’ensemble des objectifs initiaux et constitue une démonstration simple mais complète d’un jeu en Python avec Pygame.

7. Améliorations possibles

Plusieurs fonctionnalités pourraient enrichir le jeu :

- Ajouter un **menu d’accueil**
- Inclure plusieurs niveaux de labyrinthes
- Ajouter un **système de score** (temps, nombre de pas...)
- Mettre des animations (marcher, victoire)
- Ajouter des obstacles mobiles ou des objets à récupérer

8. Conclusion

Ce projet nous a permis de consolider nos compétences en Python, en particulier dans l'utilisation de Pygame et la création d'un mini-jeu complet.

Le développement du « Labyrinthe de la Princesse » nous a aidée à comprendre les principes de base des jeux 2D : affichage, gestion du temps, déplacements, collisions et interactions utilisateur.